

CO5_AT1 Constraint-Based Problem Solving

Name: C. Himaja

Reg. No: 192373035

REPORT

Title

Traveling Salesman Problem Using Backtracking and Branch and Bound

Objective

To develop a C program for solving the Traveling Salesman Problem (TSP) using Backtracking and Branch and Bound, while minimizing the total travel cost.

The solution should:

- Visit every city exactly once.
- Maintain path continuity.
- Return to the starting city.
- Minimize the total travel cost.
- Use pruning to avoid unnecessary searches.

Introduction

The Traveling Salesman Problem is a classical combinatorial optimization problem.

A salesman needs to visit a collection of cities. Each city must be visited exactly once, and after visiting all cities, the salesman must return to the starting city.

The objective is to find the tour with the minimum possible travel cost.

For n cities, there can be a very large number of possible tours. Therefore, TSP becomes computationally expensive as the number of cities increases.

Backtracking systematically explores possible tours, while Branch and Bound eliminates branches that cannot produce a better solution than the best solution already found.

Problem Statement

Given n cities and a cost matrix representing the travel cost between every pair of cities, find the minimum-cost Hamiltonian cycle.

For example:

Number of cities = 4

Cost matrix:

	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

The salesman starts at City 0, visits all cities once, and returns to City 0.

Constraints**Constraint 1 — City Visitation**

Every city must be visited exactly once.

Each city $\rightarrow$ visited once

The program uses the visited[] array to keep track of visited cities.

Constraint 2 — Path Continuity

The salesman can move from the current city to another unvisited city.

For example:

City 0 $\rightarrow$ City 1 $\rightarrow$ City 3 $\rightarrow$ City 2

Each movement must correspond to a valid edge in the cost matrix.

Constraint 3 — Return to Starting City

After visiting all cities, the salesman must return to City 0.

Therefore:

0 $\rightarrow$ 1 $\rightarrow$ 3 $\rightarrow$ 2 $\rightarrow$ 0

is a complete tour.

Constraint 4 — Cost Minimization

Among all feasible tours, the tour with the minimum total cost must be selected.

For example:

$$10 + 25 + 30 + 15 = 80$$

is better than a tour costing 95.

Why Does the Search Space Become Large?

For n cities, fixing the starting city leaves approximately:

$$(n - 1)!$$

possible tours.

For example:

Cities Approximate Tours

4	6
5	24
6	120
8	5,040
10	362,880
15	87,178,291,200

This demonstrates the rapid growth of the search space.

As the number of cities increases, checking every possible tour becomes impractical.

Backtracking Approach

Backtracking constructs the tour one city at a time.

The basic process is:

Start at City 0



Select an unvisited city



Add city to current path



Continue recursively



All cities visited?



Return to City 0



Calculate total cost



Update best solution

If a partial path cannot lead to a better solution, Branch and Bound is used to eliminate it.

Branch and Bound Pruning

Branch and Bound improves the basic Backtracking algorithm.

Before exploring a branch, the algorithm estimates the minimum possible cost of completing that path.

The program uses:

Lower Bound =

Current Cost + Remaining Cities $\times$ Minimum Edge Cost

If:

Lower Bound $\geq$ Best Known Cost

then that branch is discarded.

This is called pruning.

Example

Suppose:

Current cost = 60

Remaining cities = 3

Minimum edge cost = 10

Then:

Lower Bound = $60 + (3 \times 10)$

= 90

If the best tour already found has cost:

80

then:

$90 \geq 80$

Therefore, the branch cannot improve the current best solution and is pruned.

Algorithm

Step 1

Read the number of cities.

Step 2

Read the cost matrix.

Step 3

Set City 0 as the starting city.

Step 4

Mark the starting city as visited.

Step 5

Select an unvisited city.

Step 6

Calculate the cost of reaching that city.

Step 7

Calculate a lower bound for completing the tour.

Step 8

If the lower bound is worse than the current best cost, prune the branch.

Step 9

Otherwise, recursively continue the search.

Step 10

When all cities have been visited, return to City 0.

Step 11

Update the best tour if the new cost is smaller.

Step 12

Backtrack and explore other possible routes.

Pseudocode

TSP(currentCity, count, currentCost)

if all cities are visited:

totalCost = currentCost + cost[currentCity][start]

if totalCost < bestCost:

bestCost = totalCost

save path

return

for every unvisited city:

newCost = currentCost + cost[currentCity][city]

lowerBound =

newCost + remainingCities $\times$ minimumEdge

if lowerBound $\geq$ bestCost:

prune branch

else:

mark city as visited

add city to path

TSP(city, count + 1, newCost)

unmark city

backtrack

Optimality

The solution is optimal because the algorithm explores every branch that could potentially improve the current best solution.

Branches are pruned only when their lower-bound estimate is already greater than or equal to the best known complete tour.

Therefore, a pruned branch cannot contain a better solution.

Consequently, after the search finishes, the stored tour has the minimum possible cost.

Sample Calculation

For the given matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Consider:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

Cost:

$0 \rightarrow 1 = 10$

$1 \rightarrow 3 = 25$

$3 \rightarrow 2 = 30$

$2 \rightarrow 0 = 15$

Total:

$10 + 25 + 30 + 15$

$= 80$

Therefore:

Minimum Travel Cost = 80

Time Complexity

The TSP is an NP-Hard optimization problem.

For the basic Backtracking approach, the worst-case number of tours is approximately:

$(n - 1)!$

Therefore, the worst-case time complexity is approximately:

$O(n!)$

Branch and Bound can significantly reduce the number of branches explored in practice, but it does not change the worst-case exponential/factorial nature of the problem.

Space Complexity

The algorithm stores:

- Cost matrix
- Visited array
- Current path
- Best path
- Recursive call stack

The cost matrix requires:

$O(n^2)$

space.

The path and recursion require:

$O(n)$

space.

Therefore, overall auxiliary/storage complexity is approximately:

$O(n^2)$

because the cost matrix dominates.

NP-Hard Complexity

TSP is classified as an NP-Hard optimization problem.

The decision version asks:

Is there a tour whose total cost is less than or equal to a given value?

The number of possible tours grows factorially with the number of cities.

For example:

10 cities $\rightarrow 9! = 362,880$ tours

15 cities $\rightarrow 14! = 87,178,291,200$ tours

Therefore, an exact algorithm becomes increasingly expensive for large inputs.

Advantages

1. Finds the exact optimal tour.
2. Uses systematic Backtracking.
3. Branch and Bound reduces unnecessary exploration.
4. Suitable for small and moderate TSP instances.
5. Provides a clear demonstration of NP-Hard optimization.

Limitations

1. Computational cost increases rapidly as the number of cities increases.
2. Worst-case complexity remains approximately $O(n!)$.
3. Large real-world instances can become impractical.
4. The quality of the lower bound affects pruning performance.
5. Memory requirements increase with the cost matrix.

Real-World Applications

TSP concepts are used in many practical areas:

- Delivery route optimization
- Logistics planning
- Vehicle routing
- Salesperson travel planning
- Manufacturing
- Robotics
- Drone route planning
- Network optimization
- Warehouse operations
- Transportation planning

For large industrial problems, exact Backtracking may be replaced or combined with:

- Genetic Algorithms

- Simulated Annealing
- Ant Colony Optimization
- Integer Programming
- Approximation Algorithms
- Heuristic search

These methods trade guaranteed optimality for improved scalability.

Performance Analysis

For the sample 4-city problem:

Number of cities = 4

There are:

$$(4 - 1)! = 6$$

possible tours when the starting city is fixed.

Branch and Bound can avoid exploring some of these tours if their lower bounds are already worse than the best solution.

For larger problems, the pruning becomes increasingly important because blindly examining every permutation becomes expensive.

Result

The Traveling Salesman Problem was successfully implemented in C using Backtracking and Branch and Bound.

For the sample input, the program finds an optimal tour such as:

City 0 → City 1 → City 3 → City 2 → City 0

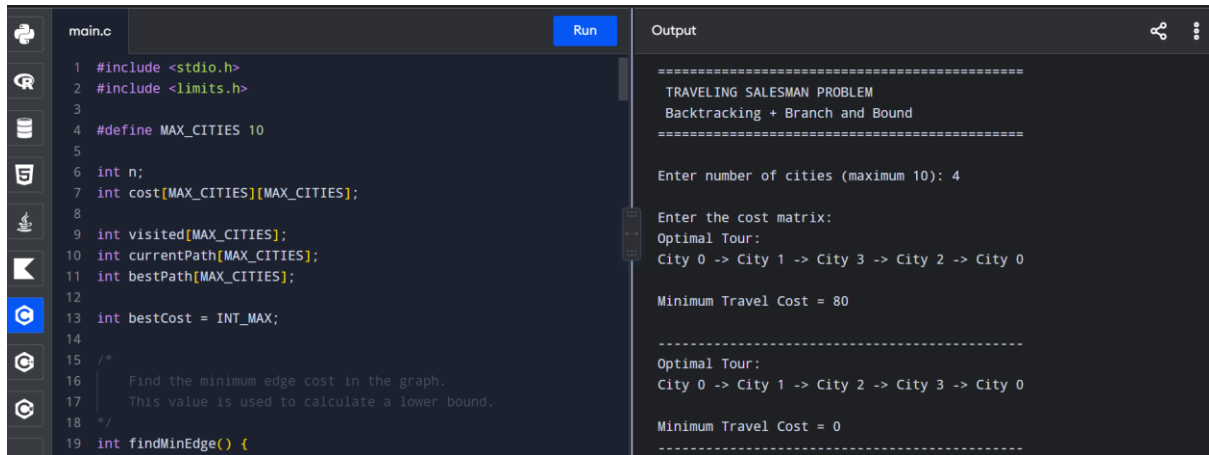
with:

Minimum Travel Cost = 80

The Branch and Bound technique eliminates paths that cannot produce a better solution. Since no potentially optimal branch is incorrectly discarded, the algorithm guarantees the optimal solution.

However, TSP remains an NP-Hard problem, and its factorial worst-case complexity makes the exact approach unsuitable for very large numbers of cities.

SCREENSHOTS TO INCLUDE



This screenshot shows the initial setup of the program. The code in `main.c` includes `<stdio.h>` and `<limits.h>`, defines `MAX_CITIES` as 10, and declares arrays for `cost`, `visited`, `currentPath`, and `bestPath`. It also sets `bestCost` to `INT_MAX`. A comment indicates the purpose of `findMinEdge()` is to find the minimum edge cost. The output window displays the program title, prompts for the number of cities (4), and shows the initial optimal tour and minimum travel cost (80).

```
1 #include <stdio.h>
2 #include <limits.h>
3
4 #define MAX_CITIES 10
5
6 int n;
7 int cost[MAX_CITIES][MAX_CITIES];
8
9 int visited[MAX_CITIES];
10 int currentPath[MAX_CITIES];
11 int bestPath[MAX_CITIES];
12
13 int bestCost = INT_MAX;
14
15 /*
16  Find the minimum edge cost in the graph.
17  This value is used to calculate a lower bound.
18 */
19 int findMinEdge() {
```

Output

```
=====
TRAVELING SALESMAN PROBLEM
Backtracking + Branch and Bound
=====

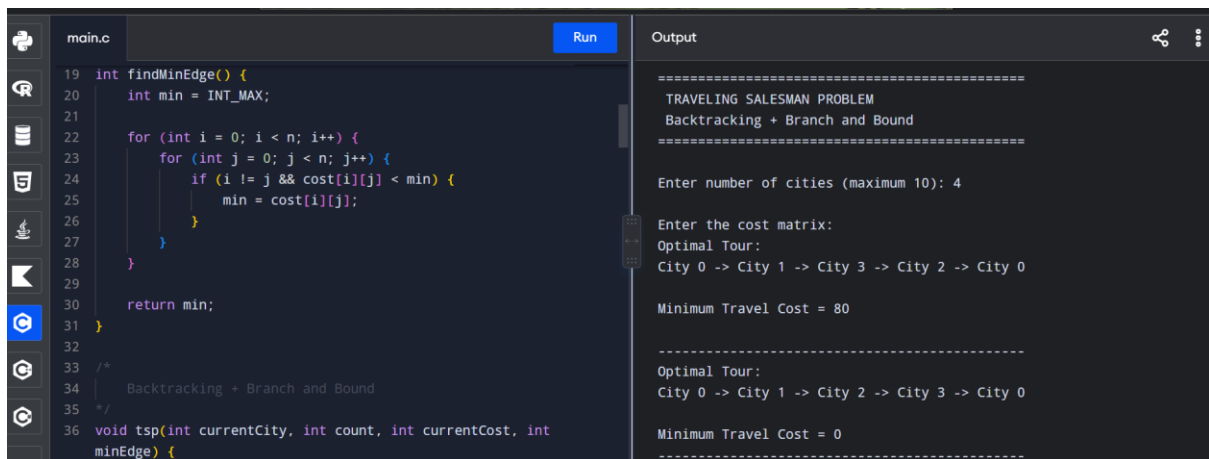
Enter number of cities (maximum 10): 4

Enter the cost matrix:
Optimal Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Travel Cost = 80

-----
Optimal Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Travel Cost = 0
-----
```



This screenshot shows the implementation of the `findMinEdge()` function. It iterates through all possible edges between cities to find the minimum cost edge that hasn't been visited. The output window remains the same as in the first screenshot.

```
19 int findMinEdge() {
20     int min = INT_MAX;
21
22     for (int i = 0; i < n; i++) {
23         for (int j = 0; j < n; j++) {
24             if (i != j && cost[i][j] < min) {
25                 min = cost[i][j];
26             }
27         }
28     }
29
30     return min;
31 }
32
33 /*
34  Backtracking + Branch and Bound
35 */
36 void tsp(int currentCity, int count, int currentCost, int
minEdge) {
```

Output

```
=====
TRAVELING SALESMAN PROBLEM
Backtracking + Branch and Bound
=====

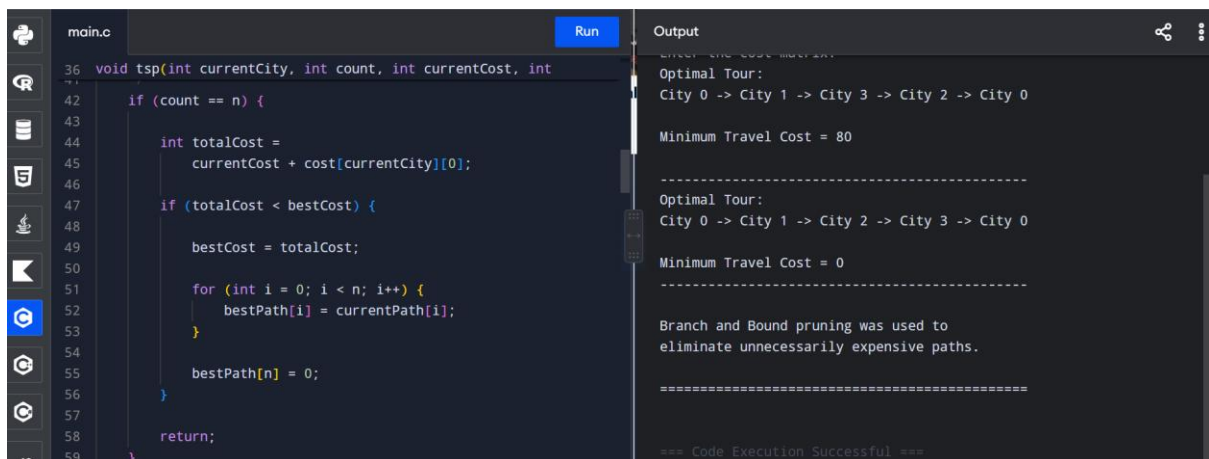
Enter number of cities (maximum 10): 4

Enter the cost matrix:
Optimal Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Travel Cost = 80

-----
Optimal Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Travel Cost = 0
-----
```



This screenshot shows the implementation of the `tsp` function. It calculates the total cost of the current path, updates the best cost and path if a better solution is found, and uses backtracking to explore other paths. The output window shows the final optimal tour and minimum travel cost (80), along with a message indicating that branch and bound pruning was used.

```
36 void tsp(int currentCity, int count, int currentCost, int
minEdge) {
37
38     if (count == n) {
39
40         int totalCost =
41             currentCost + cost[currentCity][0];
42
43         if (totalCost < bestCost) {
44
45             bestCost = totalCost;
46
47             for (int i = 0; i < n; i++) {
48                 bestPath[i] = currentPath[i];
49             }
50
51             bestPath[n] = 0;
52         }
53     }
54
55     return;
56 }
57
58
59 }
```

Output

```
=====
TRAVELING SALESMAN PROBLEM
Backtracking + Branch and Bound
=====

Optimal Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Travel Cost = 80

-----
Optimal Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Travel Cost = 0
-----

Branch and Bound pruning was used to
eliminate unnecessarily expensive paths.

=====

=== Code Execution Successful ===
```